

The revision file :

1. What is Parallel Programming?

Parallel programming is the use of computing resources that work together to provide more computational power in order to solve a problem within a reasonable amount of time. It is mainly applied to problems that require large amounts of computing power for short periods. High Performance Computing (HPC) systems range from simple workstations up to the largest supercomputers.

2. What is the difference between Serial Computing and Parallel Computing?

- **Serial Computing:**
 - Traditionally, software was written for serial execution.
 - Each problem is broken into a sequence of instructions.
 - Instructions are executed one after another on a single CPU.
 - Only one instruction executes at any given time.
 - Complex problems take a long time to process.
- **Parallel Computing:**
 - Uses multiple processors simultaneously to solve a computational problem.
 - The problem is divided into discrete parts that can be solved concurrently.
 - Each part is further broken down into instructions.
 - Instructions from each part execute simultaneously on different CPUs.
 - A unified control mechanism coordinates the entire process.
 - Benefits: time and cost savings, and the ability to solve larger problems.

3. What is Distributed Computing? How does it differ from Parallel Computing?

Distributed computing distributes components across multiple networked computers. These computers communicate and coordinate their actions using a unified messaging mechanism to achieve a common goal. **Benefits:** easier resource sharing, balanced load, and running programs on the most suitable machines. **Difference:**

- Parallel computing may use shared or distributed memory.
- Distributed computing always uses separate memory for each computer.

4. What is Grid Computing and Cloud Computing?

- **Grid Computing:** A special type of distributed computing. It aggregates computing resources across a local network or even the Internet, making users feel they have access to a virtual supercomputer.
- **Cloud Computing:** A revolutionary model of infrastructure sharing over the Internet. It allows sharing of massive computational power and resources on demand.

5. What are the four steps of Foster's Parallel Algorithm Design Methodology?

1. **Partitioning (Decomposition):** Divide the problem into tasks.
2. **Communication:** Define how tasks exchange information.
3. **Agglomeration:** Reduce the number of tasks to minimize communication overhead.
4. **Mapping:** Assign tasks to processors efficiently.

6. Define the following terminologies:

- **Core:** A single computing unit with its own independent control.
- **Multicore:** A processor with several cores that can access the same memory concurrently.
- **Task:** A portion of computation that can be executed in parallel.
- **Parallelism challenge:** Finding enough parallelism is critical for high performance (Amdahl's Law).

7. What are the main performance metrics in parallel computing?

- **Execution Time:** Time elapsed between the start and end of execution.
- **Speedup (S):** Ratio between serial execution time and parallel execution time. $S = T_s / T_p$.
- **Efficiency:** Ratio of speedup to the number of processors. $E = S / N$.
- **Scalability:** How performance changes when both problem size and number of processors increase.

8. Explain Amdahl's Law.

Amdahl's Law quantifies the maximum speedup achievable by parallelizing a program, considering the fraction that cannot be parallelized. Formula:

$$S = 1 / (1 - P + P \cdot N)$$

Where:

- S: speedup,
- P: fraction of program parallelizable,
- N: number of processors. It sets realistic expectations for performance improvement.

9. What are the sources of overhead in parallel programs?

- **Inter-process communication:** Time spent exchanging data between processors.
- **Idling:** Processors may remain idle due to load imbalance, synchronization, or serial components.
- **Synchronization:** Necessary to manage shared resources and prevent data races.
- **Parallelism rejection:** Some sequential algorithms are difficult or impossible to parallelize.

10. What are the main bottlenecks in parallel computing systems?

- **Memory latency:** Delay between requesting data and receiving it.
- **Bandwidth:** Rate at which data can be transferred to the processor. Architectural innovations (e.g., multiple data paths) have been introduced to reduce these bottlenecks.

11. What is the difference between implicit and explicit parallel programming?

- **Implicit Parallelism:**
 - Parallelism is managed automatically by the system or framework.
 - Example: OpenMP, Java Fork-Join, Python concurrent.futures..
 - Easier for programmers but less control and sometimes lower efficiency.
- **Explicit Parallelism:**
 - Programmer explicitly defines parallelism, synchronization, and communication.
 - Example: MPI.
 - Provides full control and efficiency but is more complex.

12. What are the advantages and disadvantages of implicit parallel programming?

- **Advantages:** Programmer does not worry about task division or communication; easier design.
- **Disadvantages:** Less control, sometimes lower efficiency, debugging can be difficult.

13. What are the advantages and disadvantages of explicit parallel programming?

- **Advantages:** Full programmer control, highly efficient code when done correctly.
- **Disadvantages:** More difficult to program, requires careful planning of task division and synchronization.

14. What is Concurrency? What are its levels?

Concurrency means two or more events occur within the same time interval. Tasks may execute concurrently on a single processor (via context switching) or on multiple processors. **Levels of concurrency:**

- Instruction level
- Subroutine level
- Object level
- Application level

15. What is OpenMP? What does it provide?

OpenMP is an API for multi-threaded parallelization. It provides:

- Source code directives (#pragma).

- Functions.
- Environment variables. It supports shared-memory parallelism with constructs for parallel regions, work sharing, synchronization, and scheduling.

16. What are the main OpenMP directives and clauses?

- **Directives:** `parallel`, `sections`, `for`, `reduction`, `schedule`.
- **Clauses:**
 - `if` (conditional parallelization).
 - `num_threads` (number of threads).
 - `private`, `firstprivate`, `shared` (data handling).
 - `reduction` (combining results).
 - `schedule` (distribution of loop iterations).

17. What are the scheduling types in OpenMP?

- **Static:** Predefined contiguous chunks assigned to threads.
- **Dynamic:** Chunks assigned dynamically as threads become available.
- **Guided:** Exponentially decreasing chunk sizes.
- **Runtime:** Determined by environment variable at runtime.

18. What synchronization constructs does OpenMP provide?

- **Barrier:** All threads wait until all reach the barrier.
- **Master:** Code executed only by master thread.
- **Single:** Code executed by one thread; others wait.
- **Critical:** Mutual exclusion for shared variables.
- **Atomic:** Mini critical section for single memory updates.
- **Ordered:** Ensures iterations execute in serial order.

19. What are the main MPI functions for process management and communication?

- **Initialization/Finalization:** `MPI_Init`, `MPI_Finalize`.
- **Process info:** `MPI_Comm_size`, `MPI_Comm_rank`.
- **Point-to-point communication:** `MPI_Send`, `MPI_Recv`.
- **Collective operations:** `MPI_Barrier`, `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_Reduce`, `MPI_Allreduce`.

20. How is matrix multiplication implemented in MPI?

- **Data distribution:** Scatter rows of matrix A, broadcast matrix B.
 - **Computation:** Each process computes its assigned rows.
 - **Result gathering:** Gather results back to root process. This ensures parallel execution across distributed memory systems.
-

Lecture 1 – Introduction

Q1. What are the main characteristics of problems solved by parallel programming?

- Long computation time.
- Require large resources (memory, processors).
- Often need multiple runs.
- Time-critical problems.

Q2. What is the difference between task parallelism and data parallelism?

- **Task parallelism:** Different tasks assigned to different cores.
- **Data parallelism:** Same operation performed on different portions of data.

Lecture 2 – Terminology & Metrics

Q3. Define Execution Time, Speedup, Efficiency, and Scalability.

- Execution time: elapsed time of program run.
- Speedup: $S = T_s / T_p$.
- Efficiency: $E = S / N$.
- Scalability: performance when both workload and processors increase.

Q4. What are the main sources of overhead in parallel programs?

- Communication delay.
- Idling due to imbalance.
- Synchronization cost.
- Difficulty in parallelizing best sequential algorithms.

Lecture 3 – OpenMP

Q5. What is Concurrency? List its layers.

- Concurrency: tasks executing in same time interval.
- Layers: Instruction level, Subroutine level, Object level, Application level.

Q6. What is the difference between Section directive and For directive?

- **Section:** divides work into independent tasks (functional parallelism).
- **For:** divides iterations of a loop (data parallelism).

Q7. What is the Reduction clause in OpenMP?

- Combines private copies of a variable into one global result using an operator (+, *, min, etc.).

Lecture 4 – MPI

Q8. What is MPI and why is it important?

- MPI: Message Passing Interface for distributed memory systems.
- Provides explicit communication between processes.
- Enables scalability across multiple machines.

Q9. What are MPI collective operations?

- Barrier, Broadcast, Scatter, Gather, Reduce, Allreduce.
- Used for synchronization and data distribution/collection.

Lecture 5 – Flynn’s Taxonomy

Q10. State Flynn’s classification and give examples.

- SISD: Single instruction, single data (traditional PCs).
- SIMD: Single instruction, multiple data (GPUs, vector processors).
- MISD: Multiple instruction, single data (rare, fault tolerance).
- MIMD: Multiple instruction, multiple data (clusters, multicore systems).

Q11. What is the cache coherence problem?

- In multiprocessors, each core may cache same data.
- Updates in one cache may cause inconsistency.
- Solved by invalidate/update protocols.

Lecture 6 – Parallel Algorithm Paradigms (Part 1)

Q12. Explain divide-and-conquer paradigm with example.

- Break problem into subproblems, solve concurrently, combine results.
- Example: sum of N numbers → parallel reduction tree.

Q13. How can quicksort be parallelized?

- Partition list by pivot.
- Each processor handles sublists.
- Recursively applied until sorted.

Lecture 7 – Parallel Algorithm Paradigms (Part 2)

Q14. What is data parallelism? Give example.

- Partition data among processors, each performs same operation.

- Example: matrix-vector multiplication (each processor handles rows).

Q15. What is the Owner Computes Rule?

- The process that owns a data item is responsible for computing results related to it.
- Reduces communication overhead.

Q16. What is exploratory decomposition?

- Problem decomposed while exploring state space.
- Example: solving puzzles, game search.

Q17. What is speculative decomposition?

- Used when dependencies are unknown.
- Multiple branches computed speculatively until correct path is known.

Q18. Why is hybrid decomposition sometimes necessary?

- Mixing paradigms (data + recursive) improves concurrency and load balance.
-

Part A – Multiple Choice (50 Questions)

Q1. Speedup in parallel computing is defined as:

- a. Parallel time / Serial time
- b. Number of processors used
- c. Serial time / Parallel time ✓
- d. Efficiency of the program

Q2. Which of the following is an implicit parallel programming model?

- a. MPI
- b. OpenMP ✓
- c. Pthreads
- d. CUDA

Q3. In multicore processors, a key characteristic is:

- a. Each core has separate memory
- b. Cores can access the same memory concurrently ✓
- c. Only one core active at a time
- d. Cores operate on different problems only

Q4. A system is weakly scalable if:

- a. Execution time decreases with more processors
- b. Speedup proportional to processors
- c. Execution time remains roughly constant as workload and processors increase ✓
- d. Runs only on small processors

Q5. A “Core” is defined as:

- a. A group of processors
- b. A single computing unit with independent control ✓
- c. Memory controller part
- d. Shared processor cluster

Q6. Technique allowing multiple instructions per cycle:

- a. Pipelining
- b. Superscalar execution ✓
- c. Multithreading
- d. Hyper-threading

Q7. In OpenMP static schedule, iterations are:

- a. Grabbed dynamically
- b. Assigned in contiguous blocks ✓
- c. Decreasing exponentially
- d. Set by environment variable

Q8. Foster’s methodology step grouping tasks to reduce communication:

- a. Partitioning
- b. Communication
- c. Agglomeration ✓
- d. Mapping

Q9. In distributed computing, processors share a unified address space:

- a. True
- b. False ✓

Q10. According to Amdahl’s Law, $P = 0.7$, $N = 5$, speedup $\approx$

- a. 2.17 ✓
- b. 3.5
- c. 2.5
- d. 1.42

Q11. Which is NOT a source of overhead?

- a. Communication
- b. Idling
- c. Synchronization
- d. High processor clock speed ✓

Q12. Primary goal of parallel computing:

- a. Reduce hardware cost
- b. Solve large problems faster ✓
- c. Make programming easier
- d. Increase single CPU speed

Q13. Directive to create parallel region in OpenMP:

- a. #pragma omp for
- b. #pragma omp parallel ✓
- c. #pragma omp sections
- d. #pragma omp critical

Q14. Amdahl's Law shows speedup limited by:

- a. Number of processors
- b. Parallel portion
- c. Sequential portion ✓
- d. Clock speed

Q15. If 50% sequential, max speedup =

- a. 2 ✓
- b. 4
- c. Unlimited
- d. 0.5

Q16. In explicit parallel programming, who manages parallelism?

- a. Compiler
- b. OS
- c. Programmer ✓
- d. Hardware

Q17. Main bottleneck in modern systems:

- a. CPU execution
- b. Memory system cannot supply data fast enough ✓
- c. Lack of algorithms
- d. Languages inefficient

Q18. Key motivation for parallel computing:

- a. Make serial programs faster automatically
- b. Solve larger problems or faster ✓
- c. Reduce programming complexity
- d. Eliminate algorithms

Q19. OpenMP construct dividing loop iterations:

- a. sections
- b. single

- c. master
- d. for ✓

Q20. In OpenMP, master thread ID is:

- a. 1
- b. Last thread
- c. 0 ✓
- d. Varies

Q21. Library standardized for shared-memory parallel programming:

- a. OpenMP ✓
- b. CUDA
- c. MPI
- d. BLAS

Q22. Serial time = 1000s, parallel time = 250s on 8 nodes, speedup =

- a. 4 ✓
- b. 0.25
- c. 8
- d. 1250

Q23. Disadvantage of implicit parallel programming:

- a. Needs hardware knowledge
- b. Less control, lower efficiency ✓
- c. Only for distributed memory
- d. Harder design

Q24. Difference between concurrency and parallelism:

- a. Concurrency requires multiple processors
- b. Parallelism requires multiple processors ✓
- c. Identical terms
- d. Concurrency faster

Q25. Library standardized for distributed-memory programming:

- a. OpenMP
- b. CUDA
- c. MPI ✓
- d. BLAS

Q26. OpenMP environment variable for default threads:

- a. OMP_DYNAMIC
- b. OMP_NUM_THREADS ✓
- c. OMP_SCHEDULE
- d. OMP_NESTED

Q27. A cluster of computers is:

- a. Shared memory multiprocessor
- b. Distributed memory system ✓
- c. Single-core system
- d. Implicit system

Q28. OpenMP reduction clause used to:

- a. Distribute iterations
- b. Combine local copies into one global ✓
- c. Make variable private
- d. Execute block once

Q29. OpenMP synchronization construct for one thread at a time:

- a. barrier
- b. atomic
- c. critical ✓
- d. section

Q30. In multicore processor, all cores share same memory address space:

- a. True ✓
- b. False

Q31. Flynn's taxonomy: SISD means:

- a. Single instruction, single data ✓
- b. Single instruction, multiple data
- c. Multiple instruction, single data
- d. Multiple instruction, multiple data

Q32. SIMD problem with conditional statements is called:

- a. Divergence ✓
- b. Deadlock
- c. False sharing
- d. Latency

Q33. MISD architecture is rare because:

- a. Requires multiple data streams
- b. Few real systems exist ✓
- c. Not scalable
- d. Needs GPUs

Q34. MIMD example:

- a. GPU
- b. Cluster of workstations ✓
- c. Single-core PC
- d. Vector processor

Q35. UMA vs NUMA difference:

- a. UMA equal memory access ✓
- b. NUMA equal memory access
- c. UMA slower
- d. NUMA cheaper

Q36. Cache coherence problem occurs when:

- a. Multiple caches hold copies of same data ✓
- b. CPU too slow
- c. Memory too small
- d. Threads idle

Q37. Snoopy protocol works by:

- a. Directory lookup
- b. Monitoring bus transactions ✓
- c. Compiler directives
- d. GPU scheduling

Q38. Directory-based protocol advantage:

- a. Avoids broadcasting ✓
- b. Needs fewer processors
- c. No memory
- d. Faster clock

Q39. Divide-and-conquer example:

- a. Sum of N numbers ✓
- b. Hyper-threading
- c. Cache coherence
- d. MPI_Send

Q40. Parallel quicksort works by:

- a. Sequential pivot only
- b. Partition list, assign sublists to processors ✓
- c. Sorting one element
- d. Using cache

Q41. Data parallelism example:

- a. Matrix-vector multiplication ✓
- b. Cache coherence
- c. Directory protocol
- d. Flynn taxonomy

Q42. Owner Computes Rule means:

- a. Owner process computes results for its data ✓
- b. Master computes all

- c. GPU computes
- d. Compiler computes

Q43. Exploratory decomposition example:

- a. 15-puzzle search ✓
- b. Gaussian elimination
- c. Cache protocol
- d. MPI_Bcast

Q44. Speculative decomposition used when:

- a. Dependencies unknown ✓
- b. Data parallel
- c. UMA system
- d. SIMD

Q45. Hybrid decomposition needed when:

- a. Mix of paradigms improves concurrency ✓
- b. Only one paradigm used
- c. Cache coherence
- d. Flynn taxonomy

Q46. Mapping tasks to processes is important because:

- a. Ensures load balance and reduces idling ✓
- b. Increases clock speed
- c. Reduces memory size
- d. Eliminates synchronization

Q47. MPI collective operation to distribute data:

- a. Scatter ✓
- b. Gather
- c. Reduce
- d. Barrier

Q48. MPI collective operation to collect results:

- a. Gather ✓
- b. Scatter
- c. Bcast
- d. Barrier

Q49. MPI collective operation to combine values:

- a. Reduce ✓
- b. Scatter
- c. Gather
- d. Bcast

Q50. MPI collective operation to combine and share results to all:

- a. Allreduce ✓
 - b. Reduce
 - c. Scatter
 - d. Gather
-

Part B – True / False (20 Questions)

T1. The #pragma omp master directive has no implied barrier.

✓ True

T2. Load imbalance is a common cause of idling in parallel programs.

✓ True

T3. Efficiency in parallel computing can never exceed 1 (100%).

✓ True

T4. In implicit parallel programming, the programmer explicitly creates threads.

✗ False

T5. In a multicore processor, all cores share the same memory address space.

✓ True

T6. OpenMP is primarily designed for distributed-memory systems.

✗ False

T7. In distributed computing, each processor has its own memory.

✓ True

T8. Amdahl's Law shows that the sequential portion limits maximum speedup.

✓ True

T9. Superscalar execution allows issuing multiple instructions per cycle.

✓ True

T10. In OpenMP, the master thread always has ID = 0.

✓ True

T11. MPI is used for communication in shared-memory systems.

✗ False

T12. Barrier synchronization forces all threads to wait until all reach the barrier.

✓ True

T13. Critical sections in OpenMP allow multiple threads to update a shared variable simultaneously.

✗ False

T14. Atomic directive in OpenMP is a lightweight version of critical.

✓ True

T15. In SIMD architectures, divergence occurs when processors follow different branches.

✓ True

T16. Cache coherence problem arises when multiple caches hold inconsistent copies of data.

✓ True

T17. In NUMA systems, memory access time is uniform across all processors.

✗ False

T18. MPI collective operation “Scatter” distributes data from one process to many.

✓ True

T19. MPI collective operation “Gather” collects data from many processes to one.

✓ True

T20. Hybrid decomposition combines different paradigms to improve concurrency.

✓ True